

MOVIE TICKET BOOKING SYSTEM – MINI PROJECT REPORT

ABSTRACT:-

The Movie Ticket Booking System is a database-driven application designed to automate and simplify the process of selecting movies, viewing show timings, choosing seats, and booking tickets. Traditional counter-based systems are slow, error-prone, and lack real-time seat availability. This project addresses these limitations by implementing a structured relational database that stores users, movies, theatres, shows, seats, bookings, and payments. Core DBMS concepts such as primary/foreign keys, SQL joins, triggers, normalization, and constraints ensure accuracy, prevent double booking, and maintain real-time seat updates. The frontend interacts with the backend through APIs to provide a seamless booking experience. The project demonstrates how efficient database design improves speed, reliability, user convenience, and theatre management efficiency. It highlights the importance of DBMS in creating scalable, secure, and user-friendly reservation systems.

1. INTRODUCTION:-

Movie ticket booking has evolved significantly in the last decade. Earlier, customers had to visit theatres physically, wait in long queues, and manually purchase tickets. This method caused delays, confusion, and lacked real-time information on availability. With the rise of digital platforms, consumers now expect fast, convenient, and reliable online services.

A **Movie Ticket Booking System** full-fills this requirement by allowing users to explore movies, browse showtimes, select seats, and book tickets instantly. Such a system requires a strong database backend since every operation—adding movies, updating seat availability, processing payments—depends on accurate and real-time data management.

This project uses **Database Management System concepts (DBMS)** to develop an efficient, automated, and error-free ticket booking solution. The database handles:

- User authentication
- Movie and show management
- Seat allocation & updates
- Booking records
- Payments & transaction history

The backend is implemented using **Node.js + Express + MySQL**, while the frontend is built using **React.js**. REST APIs connect both layers to ensure proper data flow.

The main purpose of this project is to demonstrate how DBMS principles can be used to build a secure, scalable, user-friendly cinema booking platform that reduces workload for theatre staff and improves customer satisfaction.

2. LITERATURE-REVIEW:-

2.1 Digital Movie Ticketing Systems

The evolution of movie ticket booking has been widely documented in research, highlighting a clear shift from traditional counter-based reservations to efficient, digital, database-driven platforms. Modern cinema-goers increasingly rely on online systems due to their

accessibility, reduced waiting times, and ability to provide real-time updates on movie schedules and seat availability. As technology advanced, online ticketing became not only a convenience but a necessity for managing large user bases and maintaining accurate booking records.

2.2 Features of Online Booking Platforms

Studies consistently identify key features of effective ticketing platforms: structured movie listings, theatre and showtime browsing, interactive seat maps, integrated payment systems, and instant confirmation. These components form the foundation of most popular portals and directly influence user satisfaction and system reliability.

2.3 Customer Studies and User Behaviour

Customer-oriented research indicates that a majority of users—especially smartphone users—prefer online booking for its speed and simplicity. Demographic factors such as gender and education have minimal impact on adoption, whereas concerns typically arise around additional charges, payment failures, and refund processes.

2.4 UI/UX Findings

UI/UX analyses emphasize the need for clean, intuitive, and mobile-friendly designs. Simple navigation flows, minimal steps, and clear visual seat layouts significantly improve the user experience. The system developed in this project follows this recommended structure with a streamlined flow: **Home → Movie → Shows → Seats → Confirm → Pay**.

2.5 Identified Challenges

Existing literature also highlights multiple challenges in digital booking systems, including internet dependency, inconsistent UI design, potential security risks, and issues with payment reliability. The proposed system addresses several of these challenges through real-

time database updates, a structured relational schema, a clean frontend interface, and simulated secure payment logic.

In summary, the literature strongly supports the development of automated ticket booking systems, emphasizing reliability, accuracy, usability, and database integrity—all of which serve as core principles in the design and implementation of this project.

our project solves part of this through:

- A clean UI
- Real-time DB seat updates
- Simulated secure payment flow
- Normalized relational database

2.1 PROBLEM STATEMENT:-

Existing manual booking systems are slow, prone to human errors, and lack real-time seat updates. Customers face difficulties checking availability, and theatre staff struggle with record management. The project aims to create an automated database-driven system that securely stores movie, show, seat, and booking data while offering a fast, real-time user booking experience.

3. OBJECTIVES:-

- **To design and implement a normalized relational database** that reliably stores and manages information related to users, movies, shows, seat layouts, bookings, and payments.
- **To provide real-time seat availability** through efficient SQL transactions that prevent double-booking and ensure correct seat allocation.
- **To develop a user-friendly interface** enabling quick movie browsing, seat selection, booking confirmation, and secure payment completion.

- **To ensure system reliability and integrity** using SQL constraints, triggers, foreign keys, and transaction control to maintain consistent and accurate data.
- **To reduce manual work** for theatres by automating bookings.

4. METHODOLOGY:-

The system architecture comprises:

- Frontend: React application for user interaction (movie list, shows, seat selection, bookings, payment).
- Backend: Node.js + Express API handling authentication, movie management, show scheduling, seat updates, bookings, and payments.
- Database: MySQL relational database storing normalized tables for users, movies, shows, seats, bookings, and payments. Real-time updates ensure accurate seat availability using SQL constraints, joins, and JSON fields for seat lists.

4.1 FUNCTIONAL REQUIREMENTS:-

1. User Registration & Login

Allows users to create an account and securely log in to access booking features.

2.Movie Listing

Displays all available movies dynamically fetched from the database with basic details.

3.Showtime Retrieval

Shows are loaded based on the selected movie, ensuring users view accurate schedules.

4.Seat Selection

Provides a real-time interactive seat map with clear indicators for available, booked, and selected seats.

5.Booking Confirmation

Stores confirmed bookings in the database, including seat numbers saved in JSON format.

6.Payment Processing

Enables users to choose a payment mode and completes the transaction while updating booking status.

7.Edit/Delete Bookings

Allows modification or removal of existing bookings, updating the database accordingly.

4.2 NON-FUNCTIONAL REQUIREMENTS:-

1. Performance

Fast API responses and optimized SQL queries ensure smooth user interaction.

2.Security

Includes password hashing, validation, and safe database operations to prevent unauthorized access.

3.Usability

Simple, intuitive UI flow (Home → Movie → Shows → Seats → Confirm → Pay) enables easy navigation.

4.Scalability

The database design supports adding more movies, shows, or users without system redesign.

5.Reliability

Real-time seat updates and stable backend processes ensure accurate and conflict-free bookings.

6.Data Integrity

Primary/foreign keys and normalization maintain consistent, well-structured records across all tables.

5. RESULTS & OUTPUT ANALYSIS:-

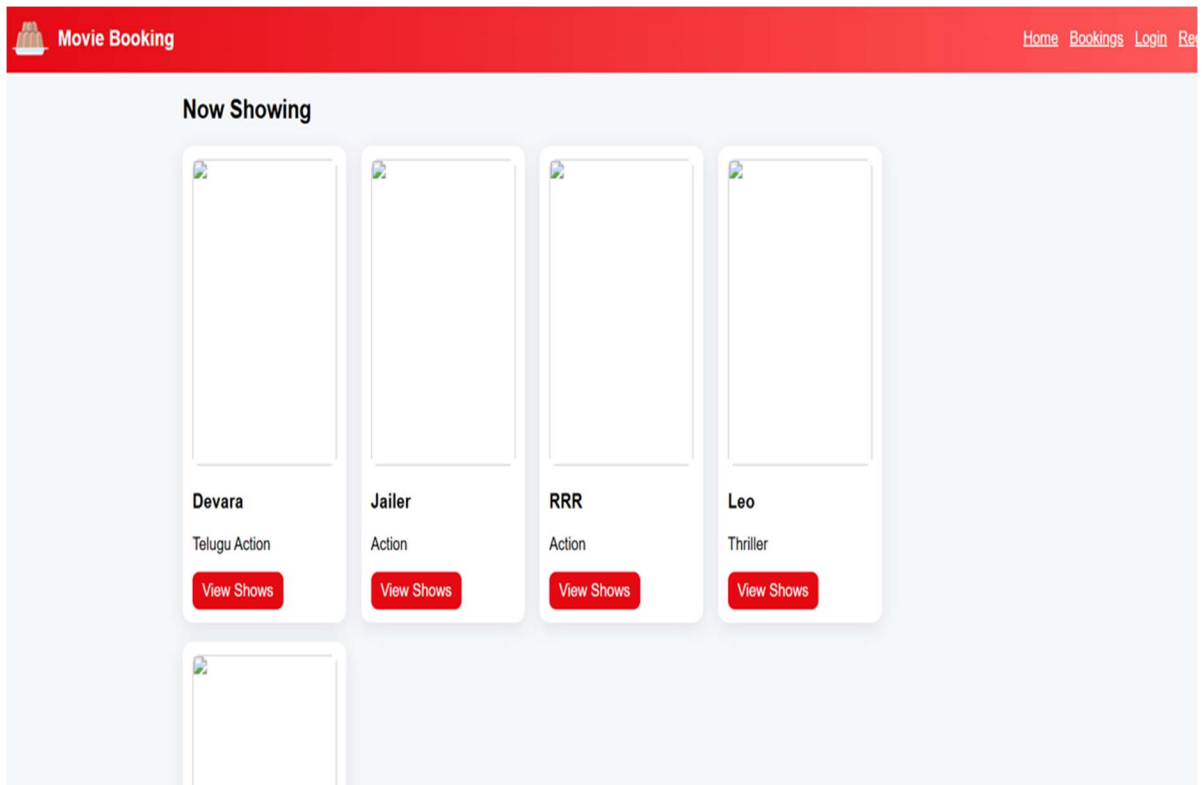
Both the backend (Node.js + Express) and frontend (React) servers are running successfully, confirming that the system is fully operational. The backend is connected to the MySQL database and ready to process API requests, while the frontend is compiled and prepared to interact with it. Together, they ensure smooth communication for registration, login, movie display, seat selection, and bookings.

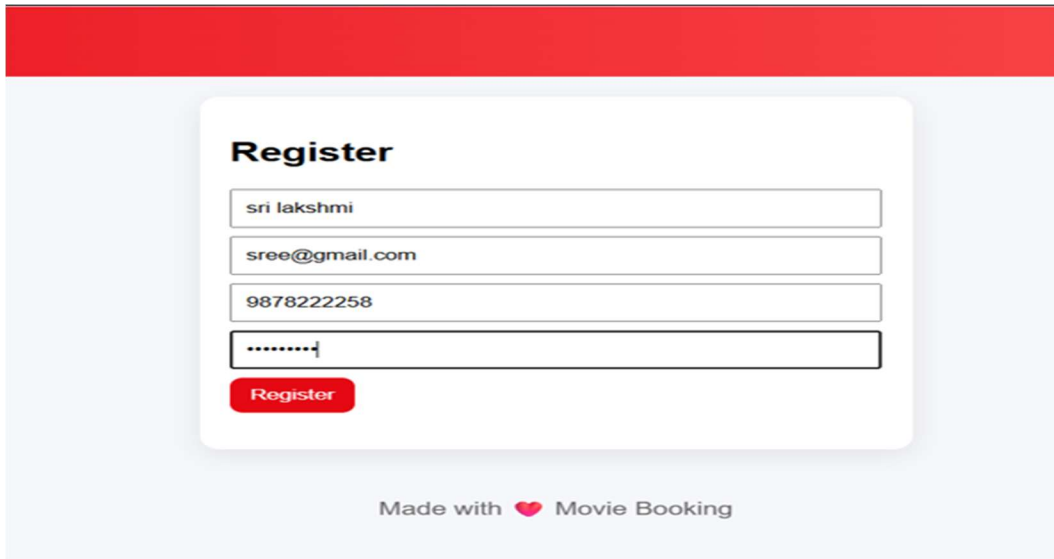
The system produced successful results:

1. User Registration Page with Homepage with Movie Listings:-

The registration page allows new users to create an account, and their details are stored in the database. After registering, the user is taken to the home page where all movies are fetched from the backend and displayed. Each movie card provides a button to view available shows.

- ❖ Movies are dynamically retrieved from the database, ensuring that the homepage always displays up-to-date film information.

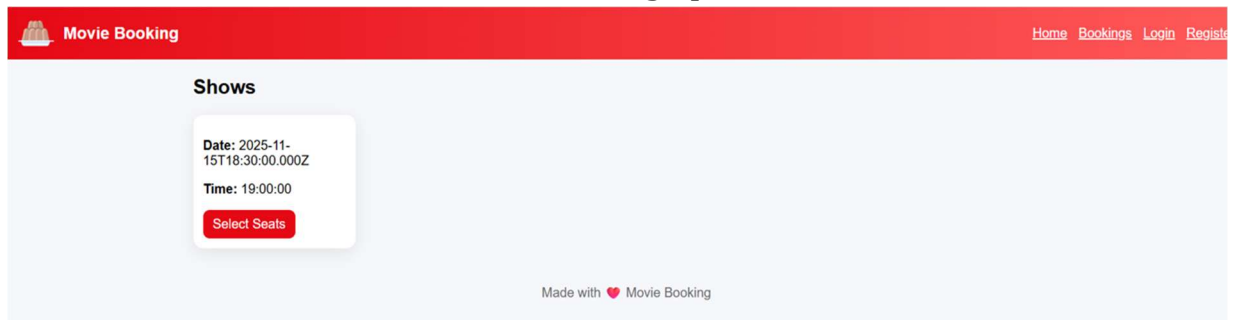




A screenshot of a web application's registration page. At the top is a solid red header bar. Below it, centered, is a white rounded rectangle containing the registration form. The form is titled "Register" in bold black text. It consists of four input fields: the first contains "sri lakshmi", the second contains "sree@gmail.com", the third contains "9878222258", and the fourth is a password field with masked characters ".....". Below the password field is a red button with the text "Register" in white. At the bottom of the white box, centered, is the text "Made with ❤️ Movie Booking".

2. Show Timings Display:-

- ❖ Show timings are fetched based on the selected movie, providing users with accurate, real-time scheduling options.



A screenshot of a web application's "Shows" page. At the top is a red header bar with a movie icon and the text "Movie Booking" on the left, and navigation links "Home", "Bookings", "Login", and "Register" on the right. Below the header, the word "Shows" is centered. In the center of the page is a white rounded rectangle containing the show details: "Date: 2025-11-15T18:30:00.000Z" and "Time: 19:00:00". Below this information is a red button with the text "Select Seats" in white. At the bottom of the page, centered, is the text "Made with ❤️ Movie Booking".

3. Seat Selection Grid:-

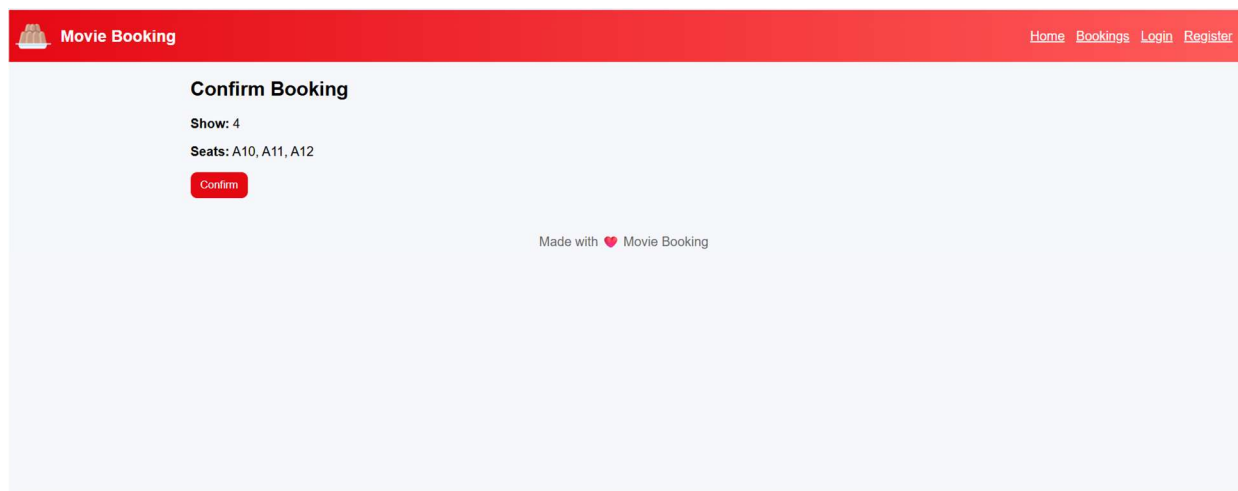
- ❖ The seat map reflects live availability, where green indicates open seats, red marks booked seats, and blue highlights the user's

current selections.



4.Booking Confirmation and Payment:-

❖ The confirmation page displays the selected show and seats for final



verification before payment. The payment page then shows the booking ID and total amount, allowing users to choose a payment mode and complete the transaction. After payment, the system updates the booking status in the database and confirms successful completion.

 **Movie Booking**

Payment

Booking ID: 11

Amount: ₹750

UPI

▼

Pay Now

Payment successful!

Made with ❤️ Movie Booking

5.Booking List with Edit/Delete:-

- ❖ The booking list displays all bookings made by the user, retrieved using SQL JOIN queries across users, shows, movies, seats, and payments. Users can update or delete bookings, and each action reflects instantly in the database. This ensures CRUD operations are working seamlessly.

Bookings					
User	Movie	Time	Seats	Actions	
sri lakshmi	RRR	19:00:00		Edit	Delete
leela	Leo	17:00:00		Edit	Delete
Meera	Jailer	16:00:00		Edit	Delete
Arya	KGF 2	21:00:00		Edit	Delete
Arya	KGF 2	21:00:00		Edit	Delete
neelambari	RRR	19:00:00		Edit	Delete
Ananya Hebbar	KGF 2	18:00:00		Edit	Delete
Geethanjali	KGF 2	21:00:00		Edit	Delete
manya	Devara	20:00:00		Edit	Delete
Jyothi	KGF 2	18:00:00		Edit	Delete
Jyothi	KGF 2	18:00:00		Edit	Delete

The system includes a functional Edit/Delete module that allows users to modify seat selections through an interactive seat grid and update bookings instantly. Successful updates are confirmed with a popup message, ensuring clear feedback. Deleting a booking first shows a confirmation alert and then displays the user's name in a success message after removal. These interactions ensure smooth CRUD operations and accurate real-time database updates.

The screenshot displays the 'Edit Booking #5' interface. At the top, a red navigation bar contains the 'Movie Booking' logo and links for 'Home', 'Bookings', 'Login', and 'Register'. The main area features a grid of 80 seats (A1 to A80) arranged in 10 rows and 8 columns. Seats A17, A25, and A33 are highlighted in red, while A33 is also outlined with a blue border. A red 'Save' button is located at the bottom left of the grid. A white confirmation popup is centered on the screen, displaying the text 'localhost:3000 says Updated!' and an 'OK' button.

Below the first screenshot, a second screenshot shows the 'Bookings' page. It features a table with the following data:

User	Movie	Time	Seats	Actions
Meera	Jailer	16:00:00		<button>Edit</button> <button>Delete</button>
Arya	KGF 2	21:00:00		<button>Edit</button> <button>Delete</button>
neelambari	RRR	19:00:00		<button>Edit</button> <button>Delete</button>
Ananya Hebbar	KGF 2	18:00:00		<button>Edit</button> <button>Delete</button>

A confirmation popup is overlaid on the table, displaying the text 'localhost:3000 says Booking deleted for: Ananya Hebbar' and an 'OK' button.

6. SQL Verifications:-

- Bookings are confirmed and stored in the database with seats in JSON format.
- Payments update booking status to 'paid'.
- SQL JOIN queries produce complete booking reports including user, movie, showtime, seats, and payment details.
- ❖ After verifying the selected show and seats, the payment step records the transaction and updates the booking status in the MySQL database. All changes—including seats in JSON format and payment status—are stored and later retrieved through SQL JOIN queries for complete booking reports.

```

-> FROM bookings b
-> JOIN users u ON b.user_id = u.user_id
-> JOIN shows s ON b.show_id = s.show_id
-> JOIN movies m ON s.movie_id = m.movie_id;

```

booking_id	user_name	movie_title	show_time	seats	payment_status
1	Jyothi	KGF 2	18:00:00	[]	paid
2	Jyothi	KGF 2	18:00:00	[]	pending
5	Ananya Hebbbar	KGF 2	18:00:00	[]	paid
4	Geethanjali	KGF 2	21:00:00	[]	paid
7	Arya	KGF 2	21:00:00	["A62", "A7"]	pending
8	Arya	KGF 2	21:00:00	["A62", "A7"]	pending
10	leela	Leo	17:00:00	["A26", "A33"]	paid
6	neelambari	RRR	19:00:00	[]	paid
11	sri lakshmi	RRR	19:00:00	["A10", "A11", "A12"]	paid
9	Meera	Jailer	16:00:00	["A17", "A1"]	pending
3	manya	Devara	20:00:00	[]	paid

```

11 rows in set (0.00 sec)

mysql> SELECT
-> m.title AS movie_title,
-> s.show_id,
-> b.booking_id,
-> u.name AS user_name,
-> JSON_EXTRACT(b.seats, '$') AS booked_seats
-> FROM bookings b
-> JOIN shows s ON b.show_id = s.show_id
-> JOIN movies m ON s.movie_id = m.movie_id
-> JOIN users u ON b.user_id = u.user_id
-> WHERE JSON_LENGTH(b.seats) > 0;

```

movie_title	show_id	booking_id	user_name	booked_seats
KGF 2	2	7	Arya	["A62", "A7"]
KGF 2	2	8	Arya	["A62", "A7"]
Leo	3	10	leela	["A26", "A33"]
RRR	4	11	sri lakshmi	["A10", "A11", "A12"]
Jailer	6	9	Meera	["A17", "A1"]

```

--> LEFT JOIN payments p ON b.booking_id = p.booking_id;

```

user_id	user_name	email	booking_id	movie_title	show_date	show_time	seats	payment_mode	paid_amount	payment_time
2	Jyothi	jyo@gmail.com	1	KGF 2	2025-11-15	18:00:00	['A62', 'A7']	UPI	500.00	2025-11-22 14:58:25
2	Jyothi	jyo@gmail.com	2	KGF 2	2025-11-15	18:00:00	['A62', 'A7']	NULL	NULL	2025-11-22 15:06:16
4	manya	man@gmail.com	3	Devara	2025-11-17	20:00:00	['A17', 'A1']	UPI	500.00	2025-11-22 15:18:25
5	Geethanjali	gee@gmail.com	4	KGF 2	2025-11-15	21:00:00	['A10', 'A11', 'A12']	UPI	500.00	2025-11-22 15:34:06
6	Ananya Hebbar	anu@gmail.com	5	KGF 2	2025-11-15	18:00:00	['A10', 'A11', 'A12']	UPI	500.00	2025-11-22 15:36:06
7	neelambari	neel@gmail.com	6	RRR	2025-11-16	19:00:00	['A10', 'A11', 'A12']	UPI	750.00	2025-11-22 20:43:23
9	Arya	ar@gmail.com	7	KGF 2	2025-11-15	21:00:00	['A62', 'A7']	NULL	NULL	2025-11-22 20:43:23
9	Arya	ar@gmail.com	8	KGF 2	2025-11-15	21:00:00	['A62', 'A7']	NULL	NULL	2025-11-22 20:43:23
10	Meera	mee@gmail.com	9	Jailer	2025-11-17	16:00:00	['A17', 'A1']	NULL	NULL	2025-11-22 20:43:23
11	leela	lee@gmail.com	10	Leo	2025-11-16	17:00:00	['A26', 'A33']	UPI	500.00	2025-11-22 20:43:23
12	sri lakshmi	sree@gmail.com	11	RRR	2025-11-16	19:00:00	['A10', 'A11', 'A12']	UPI	750.00	2025-11-22 20:43:23

11 rows in set (0.00 sec)

```

mysql> SELECT
--> m.title AS movie_title,
--> s.show_id,
--> b.booking_id,
--> u.name AS user_name,
--> JSON_EXTRACT(b.seats, '$') AS booked_seats
--> FROM bookings b
--> JOIN shows s ON b.show_id = s.show_id
--> JOIN movies m ON s.movie_id = m.movie_id
--> JOIN users u ON b.user_id = u.user_id
--> WHERE JSON_LENGTH(b.seats) > 0;

```

movie_title	show_id	booking_id	user_name	booked_seats
KGF 2	2	7	Arya	["A62", "A7"]
KGF 2	2	8	Arya	["A62", "A7"]
Leo	3	10	leela	["A26", "A33"]
RRR	4	11	sri lakshmi	["A10", "A11", "A12"]
Jailer	6	9	Meera	["A17", "A1"]

5 rows in set (0.00 sec)

```

mysql> SELECT
--> m.title,
--> COUNT(b.booking_id) AS total_bookings
--> FROM movies m
--> LEFT JOIN shows s ON m.movie_id = s.movie_id
--> LEFT JOIN bookings b ON s.show_id = b.show_id
--> GROUP BY m.movie_id;

```

title	total_bookings
KGF 2	6
Leo	1
RRR	2
Jailer	1
Devara	1

5 rows in set (0.00 sec)

6. CONCLUSION:-

The Movie Ticket Booking System successfully demonstrates the use of DBMS principles to manage ticket-booking operations efficiently. The system ensures accurate seat availability, secure booking processing, and organized data handling through normalization and SQL constraints. With an intuitive frontend and strong backend architecture, it delivers a reliable and scalable solution suitable for real-world cinema ticketing. The system can be enhanced further with features such as QR-based entry, admin dashboards, and real payment gateway integration.

7.REFERENCES:-

1. IRJMETs Research Article on Movie Ticket Booking Systems.
2. "Customer Opinion Study of Online Ticket Booking" – Academic Survey Paper.
3. UI/UX Design Principles for Movie Booking Applications.
4. Project Report Documents (Google Shared Links).
5. Journal articles on DBMS implementation in reservation systems.